

# Adaptive Retrieval-Augmented Generation Using Query Complexity and Context Optimization

Aniruddh Batwal and Manisha Bharadwaj  
Department of Computer Science and Engineering  
Vellore Institute of Technology  
Chennai, India

Emails: aniruddhbatwal@gmail.com, manisha14bst@gmail.com

**Abstract**—Retrieval-Augmented Generation (RAG) systems typically apply a fixed retrieval policy to every query, regardless of the query’s inherent complexity. This uniform approach may retrieve excessive context for simple factual questions or insufficient evidence for complex multi-hop queries. In this work, we present AdaptiveRAG, a query-complexity-aware RAG pipeline that dynamically selects retrieval depth and conditionally applies context compression. A fine-tuned DistilBERT classifier routes queries as either SIMPLE ( $K = 2$ , no compression) or COMPLEX ( $K = 10$ , LLMingua-2 chunk-wise compression). We compare DistilBERT against a BERT alternative, finding that DistilBERT was better suited to this experimental setup based on higher accuracy, recall, F1, and lower routing latency. We evaluate AdaptiveRAG against three baselines on a controlled 10-query held-out end-to-end benchmark drawn from PopQA and HotpotQA. The router achieves 99.33% accuracy on a 298-sample router evaluation artifact with 12.55 ms average inference latency. On the benchmark, all strategies yield EM = 0.0 due to the limited SmoLM-135M-Instruct generator, with Token F1 below 0.007. AdaptiveRAG achieves the lowest total latency among retrieval-augmented strategies (16.514 s vs. 18.472 s for Fixed RAG). A compression ablation demonstrates 51.46% token reduction with a net latency improvement of 8.806 s, as generation savings (21.419 s) exceed compression overhead (13.488 s) under CPU-based inference. These results demonstrate the feasibility of adaptive routing while highlighting that answer-quality conclusions require a stronger generator.

**Index Terms**—retrieval-augmented generation, query complexity, adaptive retrieval, context compression, LLMingua, dense retrieval, question answering

## I. INTRODUCTION

Retrieval-Augmented Generation (RAG) has emerged as a foundational paradigm for knowledge-intensive NLP tasks [1], [2]. By conditioning generation on retrieved evidence, RAG systems access knowledge beyond their parametric memory.

A common RAG design retrieves a fixed number of passages for every query. Such a one-size-fits-all policy ignores variation in evidence requirements. A short factual query may be answerable from a small amount of evidence, whereas a multi-hop question may require several relevant passages. In addition, increasing context length raises downstream processing cost, while prompt compression can reduce context size only by adding a separate compression step. Therefore, the overall benefit of a RAG policy depends jointly on retrieval depth, context size, compression cost, and generator behavior.

Prior research has studied important pieces of this design space. Adaptive-RAG uses question complexity to select

different retrieval strategies [3]. LLMingua and LLMingua-2 study prompt compression as a mechanism for reducing the amount of information processed by downstream language models [4], [5]. Dense retrieval methods such as DPR provide a basis for semantic retrieval from large text collections [6]. These works motivate a system-level question: can a lightweight complexity decision be used to select both retrieval depth and whether compression is worthwhile?

This paper presents AdaptiveRAG, a research prototype that investigates this question under controlled CPU-only execution. The work does not claim novelty for DistilBERT routing, dense embeddings, ChromaDB, or LLMingua-2 individually. Instead, its contribution is an empirical and system-level investigation of a deterministic policy that couples query-complexity routing with adaptive retrieval and selective compression.

The evaluation compares four strategies under a shared experimental infrastructure. The study explicitly reports the limitations of the chosen small generator and the small held-out benchmark rather than interpreting the observed answer-quality values as evidence of broad superiority. The main contributions are:

- a modular complexity-aware RAG pipeline with two deterministic execution paths;
- selective chunk-wise LLMingua-2 compression for the deep-retrieval path;
- controlled comparison against LLM-only, Fixed RAG, and Always-Compress baselines;
- a compression ablation that quantifies token reduction and latency economics; and
- an explicit error and limitation analysis covering generator quality, label construction, compression scaling, and benchmark size.

## II. BACKGROUND AND RELATED WORK

### A. Retrieval-Augmented Generation

Lewis et al. introduced RAG by combining a pretrained sequence-to-sequence generator with non-parametric memory obtained through retrieval [1]. REALM similarly demonstrated that external document retrieval can provide a modular source of knowledge during language-model processing [2]. These approaches established the general principle of separating

model parameters from an externally indexed knowledge source.

### B. Query-Complexity-Aware Retrieval

Mallen et al. studied when parametric knowledge is insufficient and found that retrieval can be particularly useful for less popular factual knowledge [7]. Jeong et al. proposed Adaptive-RAG, where question complexity is used to choose among retrieval strategies [3]. AdaptiveRAG in this paper is narrower: the router is a binary classifier and its output directly controls retrieval depth and compression. The purpose is not to reproduce the original Adaptive-RAG strategy, but to test a related complexity-aware policy in a controlled local pipeline.

### C. Context Compression

LLMLingua introduced prompt compression methods designed to reduce input length while retaining information important for downstream tasks [4]. LLMLingua-2 reformulates task-agnostic prompt compression using token classification with a bidirectional encoder architecture [5]. AdaptiveRAG uses LLMLingua-2 only on its COMPLEX route. A practical implementation constraint is that the compressor used here has a 512-token maximum sequence length, so each retrieved chunk is compressed independently before the results are recombined.

### D. Dense Retrieval and Embeddings

Dense Passage Retrieval demonstrated the utility of learned dense representations for open-domain question answering [6]. The BGE family provides compact text-embedding models; the system uses the 384-dimensional BAAI/bge-small-en-v1.5 model for semantic search [8].

### E. Datasets and Small Generators

PopQA contains open-domain factual questions, including questions about long-tail entities [7]. HotpotQA was designed around questions requiring reasoning over multiple supporting documents [9]. These properties motivate their use here as proxies for relatively simple and complex questions, respectively. The generator is SmoLLM-135M-Instruct, selected because the project targets CPU-only local execution and reproducibility [10].

### F. DistilBERT Routing

DistilBERT is a smaller Transformer encoder designed to reduce inference and resource requirements while retaining substantial language-understanding ability [11]. Its compact form makes it suitable as a dedicated routing component whose compute cost is small relative to end-to-end generation.

### G. Literature Comparison

Table I compares representative prior approaches with AdaptiveRAG. The comparison is qualitative because the cited works use different datasets, model scales, and evaluation protocols.

## III. PROBLEM FORMULATION

Let  $q$  denote an input question. The query-complexity router  $R$  produces one of two classes:

$$R(q) = \arg \max_{y \in \{\text{SIMPLE}, \text{COMPLEX}\}} p(y | q). \quad (1)$$

The routing policy maps the predicted class to execution parameters:

$$K(q) = \begin{cases} 2, & R(q) = \text{SIMPLE}, \\ 10, & R(q) = \text{COMPLEX}, \end{cases} \quad (2)$$

and

$$C(q) = \begin{cases} 0, & R(q) = \text{SIMPLE}, \\ 1, & R(q) = \text{COMPLEX}. \end{cases} \quad (3)$$

The retriever returns the top- $K$  chunks

$$D_K = \text{Retrieve}(q, K(q)). \quad (4)$$

When  $C(q) = 1$ , the chunks are compressed independently:

$$D'_K = \text{Compress}(D_K, q). \quad (5)$$

The generator then computes the final answer using either  $D_K$  or  $D'_K$  as the context.

For a run, end-to-end latency is decomposed as

$$T_{\text{total}} = T_{\text{router}} + T_{\text{retrieval}} + T_{\text{compression}} + T_{\text{generation}}, \quad (6)$$

where inapplicable terms are zero.

The evaluation addresses five research questions:

- 1) How accurately can a lightweight classifier separate the proxy SIMPLE and COMPLEX query groups?
- 2) What latency patterns result from adapting retrieval depth?
- 3) How much context can chunk-wise compression remove?
- 4) Under what local execution conditions does compression cost less than the downstream generation time it saves?
- 5) What quality-efficiency behavior is observed when the complete AdaptiveRAG pipeline is compared with fixed baselines?

## IV. PROPOSED ADAPTIVERAG METHOD

### A. System Architecture

Fig. 1 shows the deterministic inference pipeline. The classifier output directly determines which retrieval policy is executed.

### B. Query Preprocessing

The preprocessor validates that the query is non-empty and performs basic text normalization such as whitespace stripping. The original query is retained for experiment logging and evaluation. Preprocessing does not make routing decisions.

TABLE I  
COMPARISON OF RELEVANT LITERATURE AND RAG SYSTEMS

| Paper / Work              | Year | Core Model / Router            | Compression / Retrieval                         | Key Difference from AdaptiveRAG                                                                                                        |
|---------------------------|------|--------------------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| RAG [1]                   | 2020 | RAG generator + retriever      | Retrieval; no compression                       | Fixed retrieval-augmented generation without query-complexity routing.                                                                 |
| Adaptive-RAG [3]          | 2024 | T5-based complexity classifier | Multiple retrieval strategies                   | Uses multi-level complexity routing rather than our binary policy and does not add the same conditional compression mechanism.         |
| LLMLingua-2 [5]           | 2024 | BERT-based token classifier    | Prompt compression                              | Focuses on compression rather than query-complexity-aware retrieval policy selection.                                                  |
| Self-RAG [12]             | 2023 | LLM self-reflection            | Retrieval with critique/reflection              | Uses generation-time self-reflection instead of a lightweight pre-retrieval router.                                                    |
| CRAG [13]                 | 2024 | Retrieval-quality evaluator    | Retrieval correction / knowledge refinement     | Evaluates retrieval quality after retrieval rather than estimating query complexity before retrieval.                                  |
| <b>AdaptiveRAG (Ours)</b> | 2026 | <b>DistilBERT (binary)</b>     | <b>Adaptive depth + conditional LLMLingua-2</b> | Combines lightweight pre-retrieval complexity routing, adaptive $K$ , and selective chunk-wise compression in one controlled pipeline. |

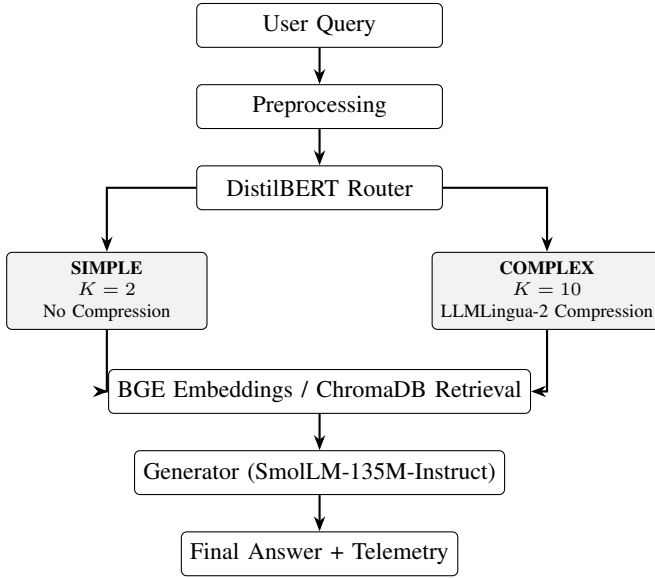


Fig. 1. AdaptiveRAG system architecture. Query complexity directly determines retrieval depth and compression activation before the shared retrieval and generation stages.

### C. Query Complexity Router

The router is a fine-tuned binary classifier. It receives the normalized query text and outputs a complexity label and confidence score. We evaluated both `distilbert-base-uncased` and `bert-base-uncased` for this role, ultimately selecting DistilBERT due to its higher recall and lower routing latency in the controlled comparison.

The original DistilBERT router was trained for three epochs with batch size 16 and learning rate  $2 \times 10^{-5}$ . The alternative BERT router was trained for two epochs with batch size 8 and the same learning rate. Both experiments used seed 42,

the same labeled data, preprocessing, evaluation data, and maximum sequence length.

Because complexity labels are inherited from dataset identity, router accuracy may partially reflect dataset-specific writing style rather than dataset-independent complexity understanding. This is discussed in Section VIII.

### D. Retrieval

BAAI/bge-small-en-v1.5 produces 384-dimensional embeddings for queries and corpus chunks. ChromaDB performs cosine-similarity search over the persistent vector index. The same corpus, embedding model, and vector index are used across all retrieval-based strategies.

### E. Selective Context Compression

Compression is enabled only for the COMPLEX route. The compressor is `microsoft/llmlingua-2-bert-base-multilingual-cased` configured with a budget of 0.5 [5].

A critical implementation correction was required during development. The initial design concatenated all retrieved chunks before compression, but  $K = 10$  commonly produced more than 512 tokens, exceeding the effective input limit of the compressor’s underlying BERT model. The final implementation compresses each retrieved chunk independently and then concatenates the compressed outputs. This respects the per-call sequence-length constraint, although compression cost scales approximately linearly with  $K$ .

When compression fails, the original context is preserved and generation continues with the uncompressed evidence. The failure is recorded in execution metadata.

### F. Answer Generation

All strategies use HuggingFaceTB/SmolLM-135M-Instruct. The common prompt is:

Question:

TABLE II  
BASELINE STRATEGIES

| Strategy        | Retrieval | $K$  | Compression | Router |
|-----------------|-----------|------|-------------|--------|
| LLM-only        | No        | 0    | No          | No     |
| Fixed RAG       | Yes       | 5    | No          | No     |
| Always-Compress | Yes       | 10   | Yes         | No     |
| AdaptiveRAG     | Yes       | 2/10 | Conditional | Yes    |

{query}

Context:  
{context}

The recorded generation configuration uses temperature 0.1, max\_new\_tokens=256, sampling enabled, and seed 42. The model was selected for CPU-only feasibility and reproducibility, not for state-of-the-art QA quality.

## V. EXPERIMENTAL SETUP

### A. Datasets and Splits

The experiments use PopQA as a relatively simple factual proxy and HotpotQA as a complex multi-hop proxy [7], [9]. The project uses 160 training samples, 40 development samples, a 298-sample router evaluation artifact used for classifier assessment, and a strictly held-out 10-query end-to-end benchmark consisting of five PopQA and five HotpotQA queries.

The 10-query benchmark was not used for router training or tuning.

### B. Corpus Construction

The corpus uses a fixed 512-token chunk size with 50-token overlap. Chunks are embedded with BAAI/bge-small-en-v1.5 and stored in a persistent ChromaDB collection using cosine similarity. The same indexed corpus is reused across retrieval-based baselines.

### C. Baselines

Four strategies are compared:

LLM-only measures generation without retrieval. Fixed RAG represents a conventional fixed retrieval policy. Always-Compress holds retrieval at  $K = 10$  and always compresses. AdaptiveRAG changes  $K$  and compression according to the router output.

### D. Fairness Controls

All four strategies use identical benchmark queries, corpus, chunking, embeddings, vector index, generator, prompt template, generation parameters, random seed, and evaluation code. The intended difference is the execution policy rather than model or infrastructure changes.

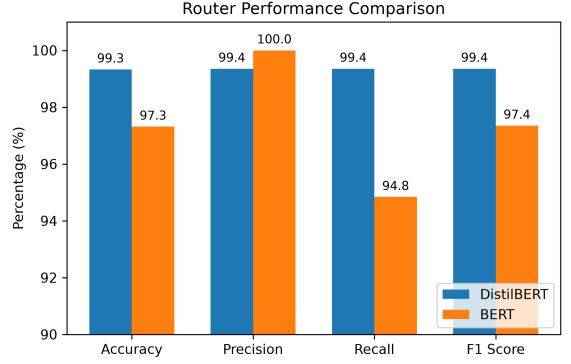


Fig. 2. Classifier performance comparison between DistilBERT and BERT on the 298-sample evaluation set.

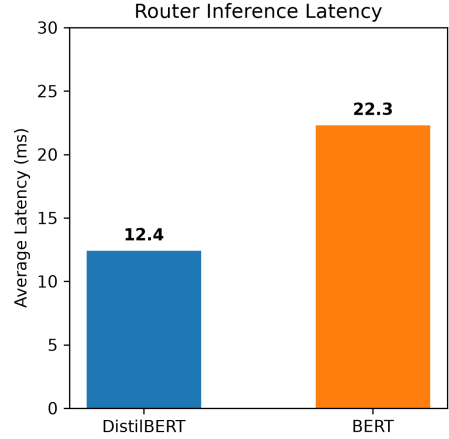


Fig. 3. Inference latency comparison of the routing classifiers.

### E. Evaluation Metrics

Answer quality is measured using Exact Match (EM) and token-level F1 after normalization of case, punctuation, articles, and whitespace. Efficiency is measured using retrieval, compression, generation, and total latency, together with context token counts and token reduction. Router quality is measured using accuracy, macro precision, macro recall, macro F1, confusion matrix, and inference latency.

## VI. EXPERIMENTAL RESULTS

### A. Classifier Comparison and Selection

To evaluate router choice, we compared fine-tuned distilbert-base-uncased and bert-base-uncased on the same 298-sample router evaluation artifact.

DistilBERT was better suited to this experimental setup based on higher accuracy, recall, F1, and lower routing latency. BERT failed to recall 8 COMPLEX queries by labeling them SIMPLE, whereas DistilBERT missed only 1. In the AdaptiveRAG policy, a COMPLEX-to-SIMPLE error reduces retrieval from  $K = 10$  to  $K = 2$  and disables compression, potentially reducing the

TABLE III  
ROUTER COMPARISON ON 298 SAMPLES

| Metric          | DistilBERT | BERT     | Difference |
|-----------------|------------|----------|------------|
| Accuracy        | 99.33%     | 97.32%   | −2.01%     |
| Precision       | 99.35%     | 100.00%  | +0.65%     |
| Recall          | 99.35%     | 94.84%   | −4.51%     |
| F1              | 99.35%     | 97.35%   | −2.00%     |
| Latency         | 12.42 ms   | 22.30 ms | +9.88 ms   |
| False Negatives | 1          | 8        | +7         |

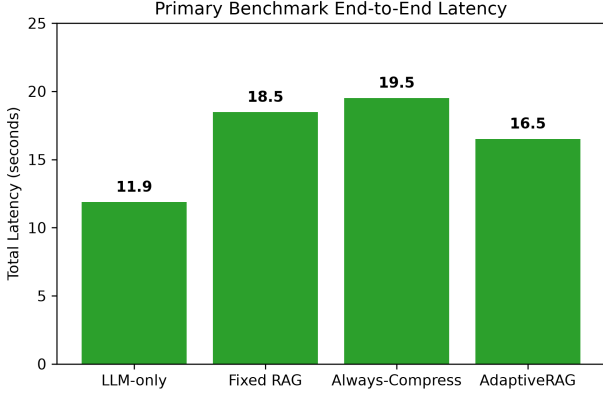


Fig. 4. Total end-to-end latency across the four benchmark strategies.

evidence available to the generator. DistilBERT was therefore selected for the primary benchmark.

### B. Primary Benchmark

The primary end-to-end benchmark uses the DistilBERT router on the ten-query held-out set.

All strategies obtain EM = 0.0, and Token F1 is extremely low. Among retrieval-augmented strategies, AdaptiveRAG achieves the lowest observed total latency at 16.514 s, compared with 18.472 s for Fixed RAG and 19.510 s for Always-Compress. These results demonstrate an efficiency pattern rather than an answer-quality advantage. The router overhead is small relative to generation latency.

### C. Downstream Classifier Effect

To measure the end-to-end impact of the alternative classifier, the AdaptiveRAG pipeline was run with the BERT router on the identical ten-query held-out benchmark. Both models made the same ten routing decisions: all five PopQA queries were classified as SIMPLE and all five HotpotQA queries as COMPLEX. Consequently, they selected the same retrieval depths and compression behavior on this small benchmark. The observed downstream quality difference is small and cannot establish a meaningful answer-quality difference on ten queries. This also shows why the larger 298-sample router evaluation is necessary for exposing routing edge cases.

TABLE IV  
PRIMARY BENCHMARK RESULTS (10 QUERIES)

| Strategy        | EM    | F1      | Ret. | Gen.  | Total |
|-----------------|-------|---------|------|-------|-------|
| LLM-only        | 0.000 | 0.00146 | 0    | 11874 | 11877 |
| Fixed RAG       | 0.000 | 0.00577 | 805  | 17661 | 18472 |
| Always-Compress | 0.000 | 0.00320 | 36   | 17890 | 19510 |
| AdaptiveRAG     | 0.000 | 0.00320 | 28   | 15679 | 16514 |

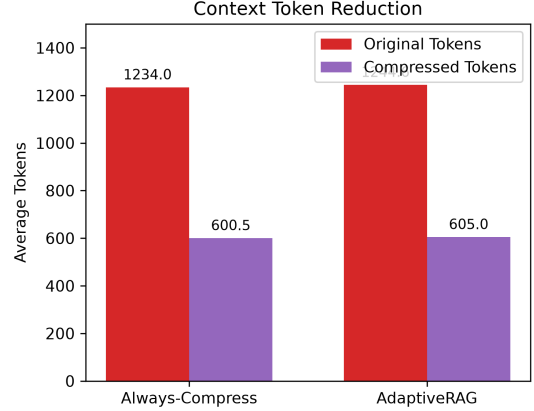


Fig. 5. Context token reduction produced by LLMlingua-2 compression.

TABLE V  
COMPRESSION ABLATION ( $K = 10$ , 10 QUERIES)

| Metric                 | No Compr. | With Compr. |
|------------------------|-----------|-------------|
| EM                     | 0.000     | 0.000       |
| Token F1               | 0.00473   | 0.00351     |
| Orig. Tokens           | —         | 1233.9      |
| Compr. Tokens          | —         | 598.9       |
| Token Reduction        | —         | 51.46%      |
| Avg. Compr. Lat. (ms)* | 0         | 13488       |
| Avg. Gen. Lat. (ms)    | 77079     | 55660       |
| Avg. Total Lat. (ms)** | 78838     | 70032       |

\* Compression latency averaged over compression-enabled queries only.

\*\* Total latency averaged over the complete benchmark run.

## VII. COMPRESSION ABLATION STUDY

To isolate the contribution of compression, a controlled ablation compares two conditions using identical  $K = 10$  retrieval, the same ten queries, and the same configuration.

Compression removes approximately 635 tokens per query on average, corresponding to a 51.46% reduction in observed context length. Generation latency decreases by 21.419 s, while compression adds 13.488 s. The measured total latency consequently decreases by 8.806 s. This favorable balance is specific to the CPU execution environment and the slow generator.

Answer quality does not improve in this ablation: EM remains zero and Token F1 decreases from 0.00473 to 0.00351. With only ten examples and a weak generator, this change is descriptive rather than statistically meaningful.

## VIII. ERROR ANALYSIS AND LIMITATIONS

### A. Generator Bottleneck

The strongest limitation is the generator. SmoLLM-135M-Instruct was selected because the project required local CPU execution, but it frequently failed to produce concise correct answers. Across all primary strategies, EM is zero and Token F1 is extremely low. Consequently, the current experiments cannot establish whether one retrieval/compression policy improves factual answer quality over another.

### B. Proxy Complexity Labels

The router labels are derived from dataset membership: PopQA is treated as SIMPLE and HotpotQA as COMPLEX. This makes the classification task easier than genuine complexity estimation and creates a risk that the router learns dataset-specific stylistic features. The 99.33% router score should therefore be interpreted as performance on the proxy classification task rather than universal complexity understanding.

### C. Small End-to-End Benchmark

The primary benchmark contains only ten queries, which is sufficient for a functional controlled demonstration but insufficient for reliable statistical significance testing or broad generalization. The identical downstream routing behavior of the two classifiers illustrates this limitation.

### D. Compression Sequence-Length Constraint

The LLMingua-2 compressor used in this implementation has a 512-token sequence-length constraint. The original concatenated compression approach was therefore invalid for many  $K = 10$  inputs. Chunk-wise compression fixes the input-length issue but may reduce globally coordinated compression and causes compression cost to scale with  $K$ .

### E. CPU-Only Execution

The reported absolute latency values are strongly affected by CPU inference. In particular, slow generation makes context reduction disproportionately valuable in the ablation. Faster hardware may produce a different balance between compression cost and generation savings.

### F. Generalizability

The measured behavior depends on the selected datasets, corpus construction, embeddings, generator, compressor, retrieval values, and hardware. The results therefore describe a controlled local prototype rather than a universal property of adaptive RAG systems.

## IX. DISCUSSION

The experiments establish three main observations. First, a lightweight classifier can add a routing decision at very small computational cost relative to the generator: the controlled classifier comparison records 12.42 ms average inference latency for DistilBERT. Second, the deterministic controller creates distinct shallow and deep execution paths, and the

shallow path is associated with lower generation latency in the ten-query AdaptiveRAG sample. Third, context compression can substantially reduce the amount of text presented to the generator, but the overall value of compression depends on the downstream generator’s cost.

The primary benchmark should be interpreted carefully. AdaptiveRAG has lower observed latency than the fixed retrieval-based baselines, but the experiment does not establish a quality improvement: all strategies obtain  $EM = 0.0$  and very low Token F1. In this environment, the main measurable effect of adaptation is therefore computational routing rather than answer-quality enhancement.

The compression ablation is also configuration-dependent. The observed 8.806 s total-latency reduction results from a specific combination of 51.46% context reduction, 13.488 s compression overhead, and 21.419 s generation-time savings. With a faster generator, the same compression process may no longer be economically attractive.

## X. CONCLUSION AND FUTURE WORK

This paper presented AdaptiveRAG, a query-complexity-aware RAG prototype that uses a lightweight classifier to select between a shallow  $K = 2$  retrieval path without compression and a deep  $K = 10$  path with LLMingua-2 compression. The system was compared with LLM-only, Fixed RAG, and Always-Compress baselines under shared infrastructure.

The router comparison shows that DistilBERT was better suited to this experimental setup than the tested BERT alternative, achieving 99.33% accuracy and 99.35% F1 with 12.42 ms average routing latency. On the ten-query held-out benchmark, AdaptiveRAG records 16.514 s average total latency versus 18.472 s for Fixed RAG and 19.510 s for Always-Compress. The compression ablation produces 51.46% token reduction and an observed 8.806 s reduction in total latency.

However, all primary strategies yield  $EM = 0.0$ , so the current experiments do not establish an answer-quality advantage for any policy. The small benchmark, proxy complexity labels, CPU-only execution, and weak generator are important limitations.

Future work includes replacing the generator with a substantially larger local model, evaluating hundreds or thousands of held-out queries, constructing complexity labels independently of dataset identity, testing on unseen datasets, sweeping  $K$  more systematically, and evaluating the pipeline on GPU hardware. Confidence-aware routing is another promising direction.

## ACKNOWLEDGMENT

The authors thank Dr. Vijayaprabakaran K, Vellore Institute of Technology, Chennai, for faculty guidance and feedback during the development of this work.

## REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

- [2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “REALM: Retrieval-augmented language model pre-training,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2020.
- [3] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, “Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity,” in *Proceedings of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.
- [4] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “LLMLingua: Compressing prompts for accelerated inference of large language models,” in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [5] Z. Pan, H. Jiang, D. Niu, and L. Qiu, “LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression,” in *Findings of the Association for Computational Linguistics (ACL)*, 2024.
- [6] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
- [7] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi, “When not to trust language models: Investigating effectiveness of parametric and non-parametric memories,” in *Proceedings of the Association for Computational Linguistics (ACL)*, 2023.
- [8] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, “C-pack: Packaged resources to advance general chinese embedding,” *arXiv preprint arXiv:2309.07597*, 2023.
- [9] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, “HotpotQA: A dataset for diverse, explainable multi-hop question answering,” in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.
- [10] L. B. Allal *et al.*, “SmolLM — blazingly fast and remarkably powerful,” HuggingFace Blog, 2024.
- [11] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “Distilbert, a distilled version of BERT: smaller, faster, cheaper and lighter,” *arXiv preprint arXiv:1910.01108*, 2019.
- [12] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-rag: Learning to retrieve, generate, and critique through self-reflection,” *arXiv preprint arXiv:2310.11511*, 2023.
- [13] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, “Corrective retrieval augmented generation,” *arXiv preprint arXiv:2401.15884*, 2024.